

# Programmieren II (Java)

## 3. Praktikum: Arrays und Strings

Sommersemester 2026

Christopher Auer



### Lernziele

- ▶ Arrays: Erstellung, Zugriff und Literale
- ▶ ☞ Strings: arbeiten mit ☞ Strings und ☞ StringBuilder
- ▶ Schauen Sie sich die Tutorial-Videos auf der *Moodle-Seite* mit *wichtigen Hinweisen* zum Praktikum und IntelliJ an!
- ▶ Sie dürfen die Aufgaben *alleine* oder zu *zweit* bearbeiten und abgeben
- ▶ Sie müssen *4 der 5* Praktika bestehen
- ▶ *Kommentieren* Sie Ihren Code
  - ▶ Jede *Methode* (wenn nicht vorgegeben)
  - ▶ *Wichtige* Anweisungen/Code-Blöcke
  - ▶ *Nicht kommentierter* Code führt zu *Nichtbestehen*
- ▶ Sie werden dazu aufgefordert werden, Ihre Lösungen *auszuführen* um die Korrektheit zu demonstrieren.
- ▶ Ab *2. Praktikum*: Alle *JUnit*-Testfälle müssen *unverändert erfolgreich durchlaufen*.
- ▶ Bestehen Sie eine Abgabe *nicht* haben Sie einen *zweiten Versuch*, in dem Sie Ihre Abgabe *verbessern müssen*.
- ▶ *Wichtig*: Sie sind einer *Praktikumsgruppe* zugewiesen, *nur* in dieser werden Ihre Abgaben *akzeptiert*!
- ▶ *Hinweis*: Die Praktika wurden mit der Unterstützung von *generativer KI* erstellt.

## Aufgabe 1: Deutschlands Energiemix

Die Daten zum weltweiten Energieverbrauch nach Quelle und Land stammen vom

- [Energy Institute – Statistical Review of World Energy \(2025\)](#)
- und wurden von [Our World in Data](#) aufbereitet.

Die Daten liegen dieser Aufgabenstellung als [CSV-Datei](#) („*comma-separated values*“) unter

SupportMaterial/energiemix/app/src/main/resources/  
energy-consumption-by-source-and-country.csv

bei. Betrachten Sie sich die ersten Zeilen dieser Datei:

```
Entity,Code,Year,Other renewables,Biofuels,Solar,Wind,...
Africa,,1965,0,0,0,0,38.626762,0,9.57193,323.49615,...
Africa,,1966,0,0,0,0,43.083344,0,10.698,...
...
Germany,DEU,1965,0,0,0,0,45.033913,0.325,30.975674,...
Germany,DEU,1966,0,0,0,0,48.70555,0.73611104,...
...
```

Bei einer CSV-Datei handelt es sich um eine **Tabelle**. Die Felder in jeder Zeile sind durch ein **Komma** (,) getrennt. Die erste Zeile zeigt die **Spaltennamen**, die restlichen Zeilen beinhalten die Daten. Jede Zeile enthält den Energieverbrauch eines Landes (oder einer Region) in einem bestimmten Jahr, aufgeteilt nach Energiequellen. Die vollständigen Spalten lauten:

Entity,Code,Year,Other renewables,Biofuels,Solar, Wind,Hydropower,Nuclear,Gas,Coal,Oil

- Entity — Name des Landes oder der Region (z.B. Germany, Africa)
- Code — **ISO-3-Ländercode** (z.B. DEU, FRA). Bei **Regionen** (z.B. Africa, Asia Pacific) ist dieses Feld **leer**.
- Year — das Jahr der Messung (1965–2024)
- Die restlichen Spalten enthalten den **Energieverbrauch** (in TWh) der jeweiligen Quelle.  
**Achtung:** Manche Felder sind **leer** (zwei aufeinanderfolgende Kommas ohne Wert dazwischen), z.B.:

```
Angola,AGO,1965,,0,,,0,,,
```

Ein leeres Feld bedeutet, dass für diese Energiequelle **keine Daten** vorliegen und als **0** behandelt werden soll.

Ihre Aufgabe ist es, aus der CSV-Datei den **Anteil erneuerbarer Energien** pro Jahr für ein bestimmtes Land zu berechnen und die Ergebnisse als Plot-Daten auszugeben.

Als **erneuerbare Energien** gelten: Other renewables, Biofuels, Solar, Wind und Hydropower. Der **Gesamtverbrauch** ist die Summe **aller** Energiequellen (einschließlich Nuclear, Gas, Coal und Oil). Der **prozentuale Anteil** erneuerbarer Energien berechnet sich als:

$$\text{Anteil (\%)} = \frac{\text{Other renewables} + \text{Biofuels} + \text{Solar} + \text{Wind} + \text{Hydropower}}{\text{Gesamtverbrauch}} \times 100$$

## Vorbereitungen

Importieren Sie das Gradle-Projekt im Verzeichnis SupportMaterial/energiemix:

- ▶ Die Java-Datei `Energiemix.java` beinhaltet bereits eine `main`-Methode, die die CSV-Datei einliest und den Inhalt als `String` an die Methode `String csvToPlot(String csv, String countryCode)` übergibt. Das zweite Argument ist der **ISO-3-Ländercode** (z.B. "DEU" für Deutschland). Ändern Sie **nicht die Implementierung** der `main`-Methode!
- ▶ Aktuell gibt die Methode einen **fixen String** in folgendem Format zurück:

```
1965 1.518143
1966 1.637798
1967 1.559359
1968 1.482081
```

Jede Zeile beinhaltet ein **Jahr** und den **prozentualen Anteil** erneuerbarer Energien in diesem Jahr (z.B. 1.52% im Jahr 1965).

- ▶ Die `main`-Methode schreibt den Inhalt des zurückgegebenen Strings in die Datei `app/energiemix.txt`.
- ▶ Führen Sie das Programm aus und prüfen Sie, ob die aktuelle Version die **vier Werte** von oben in die Datei `energiemix.txt` schreibt.

### Implementierung von `csvToPlot`

Implementieren Sie `csvToPlot`:

- ▶ Verarbeiten Sie die CSV-Datei **Zeile für Zeile**.
- ▶ **Filtern** Sie nur die Zeilen, deren Code-Spalte (zweite Spalte) dem übergebenen `countryCode` entspricht. Verwenden Sie `String.equals` für den Vergleich.
- ▶ **Parsen** Sie für jede passende Zeile die Energiewerte. Denken Sie daran: **Leere Felder** (leerer `String` nach `split`) sollen als **0.0** behandelt werden. Verwenden Sie `String.trim()` und prüfen Sie mit `String.isEmpty()`, ob ein Feld leer ist, bevor Sie `Double.parseDouble` aufrufen.
- ▶ Berechnen Sie den **Anteil erneuerbarer Energien** (in Prozent) wie oben beschrieben.
- ▶ Schreiben Sie für jedes Jahr eine Zeile der Form `Jahr Prozent` in einen `StringBuilder`.
- ▶ Geben Sie den **resultierenden String** zurück.

Wenn Sie nicht wissen, wie Sie vorgehen sollen, hier ein paar Tipps:

- ▶ Zum Vergleich, hier die berechneten Werte für Deutschland ("DEU") der ersten und letzten vier Jahre:

```
1965 1.518143
1966 1.637798
1967 1.559359
1968 1.482081
...
2021 18.082499
2022 20.329156
2023 23.602962
2024 23.968008
```

- ▶ Sie können einen `String` mit der Methode `String[] split(String pattern)` an den von `pattern` definierten Stellen auftrennen. Für **Zeilenumbrüche** verwenden Sie z.B. `"\n"` oder `"\r\n"`. Für **Kommas** verwenden Sie `","`.
- Wichtig:** Bei `split(",")` für eine Zeile wie `Angola,AGO,1965,,0,,,0,,` erhalten Sie ein `Array`, in dem leere Felder als **leere Strings** (`""`) erscheinen. Allerdings werden leere Felder **am Ende** der Zeile von `split` standardmäßig **abgeschnitten**. Verwenden Sie daher `split(",", -1)`, um **alle** Felder zu erhalten.

- ▶ `Double.parseDouble` wandelt einen String in einen `double` um.
- ▶ Die *erste Zeile* beinhaltet *keine Werte* (Spaltennamen)!
- ▶ Die Daten sind bereits nach Land und Jahr *sortiert*. Sie müssen also *keine* Sortierung vornehmen.

#### Optional: Prüfen und Plotten der Daten

☞ `gnuplot` ist ein Opensource-Projekt zum Erstellen von *Graphen/Plots*. Die Daten in `app/energiemix.txt` können mit Hilfe von `gnuplot` grafisch dargestellt werden:

- ▶ Die *Fort Hays State University* bietet eine ☞ Online-Version von `gnuplot` an. Öffnen Sie die Seite in Ihrem Browser.
- ▶ Kopieren Sie folgendes `gnuplot`-Skript in das Feld *Script*

```
set title "Anteil erneuerbarer Energien (Deutschland)"
set xlabel "Jahr"
set ylabel "Anteil erneuerbare Energien [%]"
set autoscale
set yrange [0:∗]
f(x)=b*x+c
fit f(x) "data.txt" using 1:2 via b,c
plot "data.txt" using 1:2 w lp title "Erneuerbare", \
f(x) w l title "Trend"
```

- ▶ Kopieren Sie den Inhalt von `app/energiemix.txt` in das Feld *Data*.
- ▶ Sollte alles korrekt sein, taucht rechts oben ein Graph mit dem Anteil erneuerbarer Energien auf. Sie können deutlich den *Anstieg* der erneuerbaren Energien ab ca. dem Jahr 2000 erkennen — Deutschlands *Energiewende* in Aktion!

## Aufgabe 2: Lights Out!

In dieser Übung implementieren wir das Spiel  Lights Out. Das Spiel wird auf einem  $5 \times 5$  Gitter gespielt, auf dem sich *Lichter* befinden, die entweder *an* oder *aus* sind. Ziel ist es, *alle Lichter auszuschalten*.

Das Spielprinzip ist wie folgt: Wird ein Feld *gewählt*, so wird das Licht an dieser Stelle *sowie* die Lichter in den *vier orthogonalen Nachbarfeldern* (oben, unten, links, rechts) *umgeschaltet* — d.h. aus wird an und an wird aus. Felder am Rand haben entsprechend weniger Nachbarn. Hier ein Beispiel. Die Ausgangslage:

```

  0 1 2 3 4
+--+--+--+
0 |.|*|. |*|. |
+--+--+--+
1 |*|. |*|. |*|. |
+--+--+--+
2 |.|*|*|*|. |
+--+--+--+
3 |*|. |*|. |*|. |
+--+--+--+
4 |.|*|. |*|. |
+--+--+--+

```

Dabei steht \* für ein *eingeschaltetes Licht* und . für ein *ausgeschaltetes Licht*. Klickt man nun auf das Feld in Zeile 2, Spalte 2 (die Mitte), werden *fünf Felder* umgeschaltet: (2, 2) selbst sowie (1, 2), (3, 2), (2, 1) und (2, 3):

```

  0 1 2 3 4
+--+--+--+
0 |.|*|. |*|. |
+--+--+--+
1 |*|. |.|.|*|. |
+--+--+--+
2 |.|.|.|.|.|
+--+--+--+
3 |*|. |.|.|*|. |
+--+--+--+
4 |.|*|. |*|. |
+--+--+--+

```

Am Rand werden nur die *existierenden* Nachbarn umgeschaltet. Klickt man z.B. auf die Ecke (0, 0), so werden nur die Felder (0, 0), (0, 1) und (1, 0) umgeschaltet:

```

  0 1 2 3 4
+--+--+--+
0 |*|. |.|*|. |
+--+--+--+
1 |.|.|.|.|*|. |
+--+--+--+
2 |.|.|.|.|.|
+--+--+--+
3 |*|. |.|.|*|. |
+--+--+--+
4 |.|*|. |*|. |
+--+--+--+

```

Importieren Sie das Gradle-Projekt `lightsout`. Wie immer finden Sie *JUnit-Tests* deren Inhalte noch *auskommentiert sind*.

**Die enum CellState**

Bevor wir mit der Implementierung des Spiels starten, brauchen wir die `enum CellState`, die den Zustand eines Feldes modelliert und folgendes Attribut besitzt:

| Wert | char display |
|------|--------------|
| ON   | *            |
| OFF  | .            |

Das Attribut `char display` mit dem *Getter* `getDisplay` definiert das Zeichen für die *Bildschirmausgabe*.

- Implementieren Sie `CellState` und vergessen Sie den *Getter* nicht!

*JUnit-Test:* `CellStateTest.testValues`

- Implementieren Sie die *statische öffentliche Methode* `CellState fromDisplay(char display)`, die zu `display` den entsprechenden Wert der `enum CellState` zurückgibt, bspw. `CellState.ON` für `'*'`. Für ein unbekanntes Zeichen soll eine `IllegalArgumentException` geworfen werden.

*JUnit-Test:* `CellStateTest.testFromDisplay`

- Implementieren Sie die *öffentliche Methode* `CellState toggle()`, die den *jeweils anderen Zustand* zurückgibt:

| Wert | toggle() |
|------|----------|
| ON   | OFF      |
| OFF  | ON       |

*JUnit-Test:* `CellStateTest.testToggle`

Im Folgenden implementieren wir Lights Out in einer Klasse. Dabei speichern wir den Zustand des Spielfelds in einem *zweidimensionalen Array vom Typ* `CellState[][]`. Der erste Index gibt die *Zeile*, der zweite Index die *Spalte* an. In der Ausgangslage von oben ist z.B. `board[0][1] ↔ == CellState.ON` und `board[0][0] == CellState.OFF`.

**Die Klasse LightsOut**

Deklarieren Sie die Klasse `LightsOut` und darin die *öffentliche statische Methode* `CellState[][] ↔ getExample()`, die eine vorbereitete Belegung mit folgendem Inhalt zurückgibt:

```

0 1 2 3 4
+--+--+--+
0 |.|*|. |*|. |
+--+--+--+
1 |*|. |*|. |*|. |
+--+--+--+
2 |.|*|*|*|. |
+--+--+--+
3 |*|. |*|. |*|. |
+--+--+--+
4 |.|*|. |*|. |
+--+--+--+

```

Gehen Sie bei der Implementierung von `getExample` wie folgt vor:

- Deklarieren Sie einen *zwei-dimensionalen char*-Array, dessen Belegung Sie mit Hilfe eines *Array-Literals* definieren. Wenn Sie nicht wissen, was ein *Array-Literal* ist, dann recherchieren Sie zuerst nach!
- Wandeln Sie dann den zwei-dimensionalen `char`-Array *mit Hilfe der Methode* `CellState.fromDisplay` in einen zwei-dimensionalen `CellState`-Array um.
- Geben Sie das Ergebnis zurück.

*JUnit-Test:* `LightsOutTest.testGetExample`

### Die Methode `checkBoard`

Um die Prüfung des Parameters `CellState[][] board` im Folgenden zu vereinfachen, implementieren Sie die **statische öffentliche Methode** `void checkBoard(CellState[][] board)`. Diese prüft, ob es sich bei `board` um einen gültigen **zwei-dimensionalen Array der Größe  $5 \times 5$**  handelt, bei dem **kein Eintrag** `null` ist. Wenn nicht, dann generiert `checkBoard` eine `IllegalArgumentException`. Implementieren Sie `checkBoard` und testen Sie Ihre Implementierung mit `LightsOutTest.testCheckBoard`!

### Ausgabe einer Belegung

Implementieren Sie die **statische öffentliche Methode** `void printBoard(CellState[][] board)`, die die Belegung wie bisher gezeigt (siehe oben) ausgibt. **Testen** Sie Ihre Implementierung, indem Sie `LightsOut` mit einer `main`-Methode ausstatten, in der Sie `printBoard` mit dem Rückgabewert von `getExample` aufrufen. Vergessen Sie nicht, in `printBoard` die Gültigkeit des Parameters `board` mit `checkBoard` zu prüfen!

### Zählen der eingeschalteten Lichter

Implementieren Sie die **statische öffentliche Methode** `int countLightsOn(CellState[][] board)`, die die Anzahl der eingeschalteten Lichter (d.h. Felder mit dem Zustand `CellState.ON`) auf `board` zurückgibt.

*JUnit-Test:* `LightsOutTest.testCountLightsOn`

### Umschalten von Lichtern

Implementieren Sie die **statische öffentliche Methode** `void toggle(CellState[][] board, int row, int col)`. Diese schaltet das Licht an Position `(row, col)` sowie die **vier orthogonalen Nachbarn** (oben, unten, links, rechts) um, d.h. aus `ON` wird `OFF` und umgekehrt. Felder am Rand haben entsprechend weniger Nachbarn — achten Sie darauf, dass Sie **nicht auf ungültige Felder** zugreifen!

Prüfen Sie außerdem die Argumente:

- ▶ `board` prüfen Sie mit `checkBoard`.
- ▶ `row` und `col` müssen gültige Indizes sein (0 bis 4), ansonsten wird eine `IllegalArgumentException` geworfen.

*JUnit-Tests:* `LightsOutTest.testToggleCenter`, `LightsOutTest.testToggleCorner`, `LightsOutTest.testToggleEdge`, `LightsOutTest.testToggleDoubleRevert` und `LightsOutTest.testToggleInvalidArguments`

Wenn Sie nicht weiter wissen, hier ein paar Hinweise:

- ▶ Das angewählte Feld selbst und maximal vier Nachbarn müssen umgeschaltet werden — insgesamt also bis zu **fünf Felder**.
- ▶ Für jede Nachbarposition prüfen Sie, ob sie **innerhalb des Spielfelds** liegt.
- ▶ Verwenden Sie die Methode `CellState.toggle()` zum Umschalten.



### Prüfen ob das Spiel gelöst ist

Implementieren Sie die *statische öffentliche Methode* `boolean isSolved(CellState[][] board)`, die `true` zurückgibt, wenn *alle Lichter ausgeschaltet* sind (d.h. alle Felder den Zustand `CellState.OFF` haben); ansonsten `false`.

*JUnit-Tests*: `LightsOutTest.testIsSolved` und `LightsOutTest.testIsSolvedAfterToggles`

### Interaktives Spiel in `main`

Vervollständigen Sie die `main`-Methode wie folgt: Erstellen Sie mit `getExample` eine *Anfangsbelegung*. Solange das Spiel *nicht gelöst* ist (`isSolved` liefert `false`), wiederhole:

- ▶ Geben Sie die *aktuelle Belegung* aus.
- ▶ Geben Sie die *Anzahl der eingeschalteten Lichter* aus.
- ▶ Lesen Sie eine *Zeile* und eine *Spalte* von der Eingabe ein:

```
Enter row (0-4): 2
Enter col (0-4): 2
```

- ▶ Rufen Sie `toggle` mit den eingelesenen Werten auf.
- ▶ Zählen Sie die Anzahl der *Züge* mit.

Wenn das Spiel gelöst ist, geben Sie eine *Erfolgsmeldung* aus, z.B.:

```
All lights off! Solved in 7 moves!
```

Testen Sie Ihre Implementierung, indem Sie:

- ▶ prüfen, ob Ihr Programm nach einer gültigen Folge von Zügen korrekt mit der Erfolgsmeldung beendet. Wenn Sie das Beispiel nicht lösen können, erstellen Sie ein einfacheres Beispiel und lösen dieses.
- ▶ mit *fehlerhaften Eingaben* prüfen, ob Ihr Programm richtig reagiert (d.h. *nicht abbricht* und den Nutzer zu einer *erneuten Eingabe* auffordert).

### Optional: Undo-Funktion

Implementieren Sie eine *Undo-Funktion*, mit der der Spieler den *letzten Zug rückgängig* machen kann.

Vorschlag zur Vorgehensweise:

- ▶ Bieten Sie dem Spieler vor jedem Zug die Option, u einzugeben, um den letzten Zug rückgängig zu machen.
- ▶ Da `toggle` sein eigenes Inverses ist, können Sie einen Zug rückgängig machen, indem Sie `toggle` mit denselben Koordinaten erneut aufrufen.